University of Piraeus
Department of Digital Systems

Mobile and Wireless Communications Security

ΟΝΟΜΑΤΕΠΩΝΥΜΟ	ΑΡΙΘΜΟΣ ΜΗΤΡΩΟΥ	EMAIL	ΗΜΕΡΟΜΗΝΙΑ
ELVIS AVDULI	E22001	avdulielvis@gmail.com	30/04/2026

8th Semester

Table of Contents

1. User Managment Smart contract 2

1.1 Task Description 2

1.2 Implementation Analysis 2

2. Student Voting Smart Contract 3

2.1 Task Description 3

2.2 Implementation Analysis 4

1. User Management Smart contract

1.1 Task Description

The objective of this task is to design and implement a smart contract that manages user date in a secure and controlled manner. The contract must define a struct named User, which contains the following attributes: address, name, surname, age, grade, and occupation. The occupation field distinguishes between students and teachers within the system.

The main requirement of the assignment is to enforce proper access control over the grade of each user. Access to a user's grade must be restricted so that only authorized entities can view or modify it. Specifically, access should be granted to three categories of users: the owner of the data, users with the role of teacher, and any address that has been explicitly authorized by the owner.

To achieve this, the contract must include appropriate functions that allow users to register their data, manage permissions, and securely access or update grades. The implementation should ensure that unauthorized users cannot access sensitive information or perform restricted actions, thereby maintaining the integrity and confidentiality of the stored data.

1.2 Implementation Analysis

The implemented smart contract follows the requirements of the assignment by defining a structured and secure system for managing user data. At its core, the contract introduces a struct named User, which encapsulates all required attributes, including the user's address, personal information, academic grade, and occupation. This struct serves as the fundamental data model for the contract.

User data is stored using a mapping that associates each Ethereum address with a corresponding User struct. This allows efficient storage and retrieval of user information while ensuring that each user is uniquely identified by their address. When a user calls the addUser function, their data is stored in the mapping using msg.sender as the key, guaranteeing that each user can only register or overwrite their own information.

To implement access control, the contract uses a nested mapping structure that keeps track of permissions granted by users. This mapping allows each user to define which external addresses are allowed to access their data. Through the grantAccess and revokeAccess functions, users can dynamically manage these

permissions, providing flexibility while maintaining control over their personal information.

A key component of the contract is the mechanism used to distinguish between students and teachers. This is achieved through the occupation field, which is stored as a string. The contract includes an internal function named isTeacher, which uses the keccak256 hashing function to compare the stored occupation with the string "teacher". This approach is necessary in Solidity 0.4.17, as direct string comparison is not supported.

The access control logic is enforced in both the getGrade and setGrade functions. Before performing any operation, the contract checks whether the caller satisfies at least one of the authorization conditions. Specifically, access is granted if the caller is the owner of the data, if the caller is a teacher, or if the caller has been explicitly granted permission by the owner. If none of these conditions are met, the function execution is reverted using a require statement.

The getGrade function allows authorized users to retrieve the grade of a specific user, while the setGrade function allows authorized users to modify it. This ensures that sensitive academic information is protected and only accessible to trusted parties. At the same time, the contract maintains flexibility by allowing controlled sharing of access through the permission mechanism.

Overall, the implementation satisfies the requirements of the assignment by combining structured data storage with a robust access control system. It ensures that user data is securely managed, that permissions are explicitly defined, and that unauthorized access is prevented. This design demonstrates the importance of access control in smart contract development and highlights how decentralized systems can enforce security without relying on a central authority.

2. Student Voting Smart Contract

2.1 Task Description

The objective of this task is to design and implement a smart contract that simulates a voting process for the election of a president among students. The contract must define a struct named Student, which contains basic personal information such as address, name, surname, and age.

The system must allow the registration of students, the declaration of candidates, and the execution of a voting process. Candidates can be defined either by the

contract owner (the address that deploys the contract) or by the users themselves. A key requirement of the system is to ensure the integrity of the voting process, meaning that each user is allowed to vote only once and that each vote is correctly assigned to the selected candidate.

Additionally, the contract must include appropriate functions to manage candidate registration, voting, and the calculation of the final result. The implementation should guarantee fairness, prevent unauthorized actions, and ensure that the outcome of the election accurately reflects the votes cast by participants.

2.2 Implementation Analysis

The implemented smart contract fulfills the requirements of the assignment by providing a complete and secure voting mechanism. At its core, the contract defines a struct named Student, which stores the personal information of each participant, including their address, name, surname, and age. This struct is used to represent all users who interact with the system.

Student data is stored in a mapping that associates each Ethereum address with a corresponding Student struct. This allows users to register themselves through the registerStudent function, ensuring that each participant is uniquely identified by their address.

The contract introduces a special role, the owner, which is defined as the address that deploys the contract. The owner has elevated privileges, particularly in the process of candidate registration. However, the design also allows users to register themselves as candidates, promoting decentralization and participation in the election process.

Candidate management is handled through a mapping that records whether a specific address is a valid candidate, as well as an array that stores all candidate addresses. This dual structure allows efficient validation of candidates during voting and enables iteration over all candidates when determining the winner.

The voting process is carefully controlled to ensure fairness and integrity. The contract uses a mapping named hasVoted to track whether a user has already cast a vote. When a user attempts to vote, the contract first checks this mapping. If the user has already voted, the transaction is rejected. This guarantees that each participant can vote only once, preventing double voting.

In addition, the contract verifies that the selected address corresponds to a valid candidate before accepting the vote. This ensures that votes cannot be assigned to

arbitrary or unauthorized addresses. Once these conditions are satisfied, the vote is recorded by incrementing the vote count associated with the chosen candidate.

The contract also includes a function to determine the winner of the election. This function iterates through the array of candidates and identifies the address with the highest number of votes. The result is then returned as the winner. This approach ensures that the final outcome accurately reflects the voting results.

Overall, the implementation demonstrates a secure and reliable voting system. It enforces strict validation rules, prevents double voting, and ensures that only valid candidates can receive votes. By combining structured data storage with controlled access to functions, the contract guarantees the integrity and correctness of the election process.